Visualization Tool Exploration: Streamlit

Using Fashion-MNIST as a playground for Streamlit features

This tutorial is focused on **Streamlit** and its capabilities rather than the dataset. Fashion-MNIST is simply a convenient, interesting dataset to make the widgets feel meaningful. I am sure that Streamlit has other capabilities not encapsulated in this tutorial! When in doubt, I recommend referencing the Streamlit Documentation.

I wanted to create a tutorial for Streamlit as its interactivity allows the audience to explore the visuals on their own terms. Adding in widgets like expanders can help make the product more author-driven or audience-driven depending on what the viewer prefers. I ultimatley think that Streamlit is a simple tool to create narrative digital stories. This tutorial also helps to merge both basic and genuine implementations of Streamlit's capabilities.

Overview & Dataset | Widgets & Formatting | Charts & Plotly | Forms & Validation | MNIST Exploration

Fashion-MNIST Overview

Fashion-MNIST contains **70,000 grayscale 28×28 images** across 10 clothing classes. It's perfectly balanced (6,000 train / 1,000 test per class), which makes it ideal for demonstrating visualizations without worrying about class imbalance.

Training images	Test images	Classes	Image size
60,000	10,000	10	28 × 28 px

I personally gained access to the dataset through TensorFlow/Keras:

```
from tensorflow.keras.datasets import fashion_mnist
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()
```

Streamlit Explorer

A tutorial on Streamlit's interactive capabilities using Fashion-MNIST as our dataset.

Like the sidebar feature?

> See the code!

Tabs in this app

1. Dataset Overview
2. Widgets & Formatting
3. Charts & Plotly
4. Forms & Validation
5. MNIST Exploration

The Grammar of Graphics
Choices matter! Every visualization is built from components:

- Data
- Marks (points, lines, bars)
- Encodings (position, color, size)
- Scales and coordinate systems

Deploy ⋮

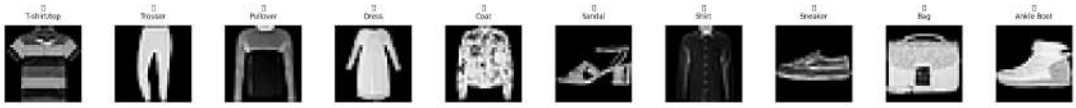
makes it ideal for demonstrating visualizations without worrying about class imbalance.

Training images	Test images	Classes	Image size
60,000	10,000	10	28 × 28 px

I personally gained access to the dataset through TensorFlow/Keras:

```
from tensorflow.keras.datasets import fashion_mnist
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()
```

Sample Images



One random sample per class. Re-run the app to see different images.

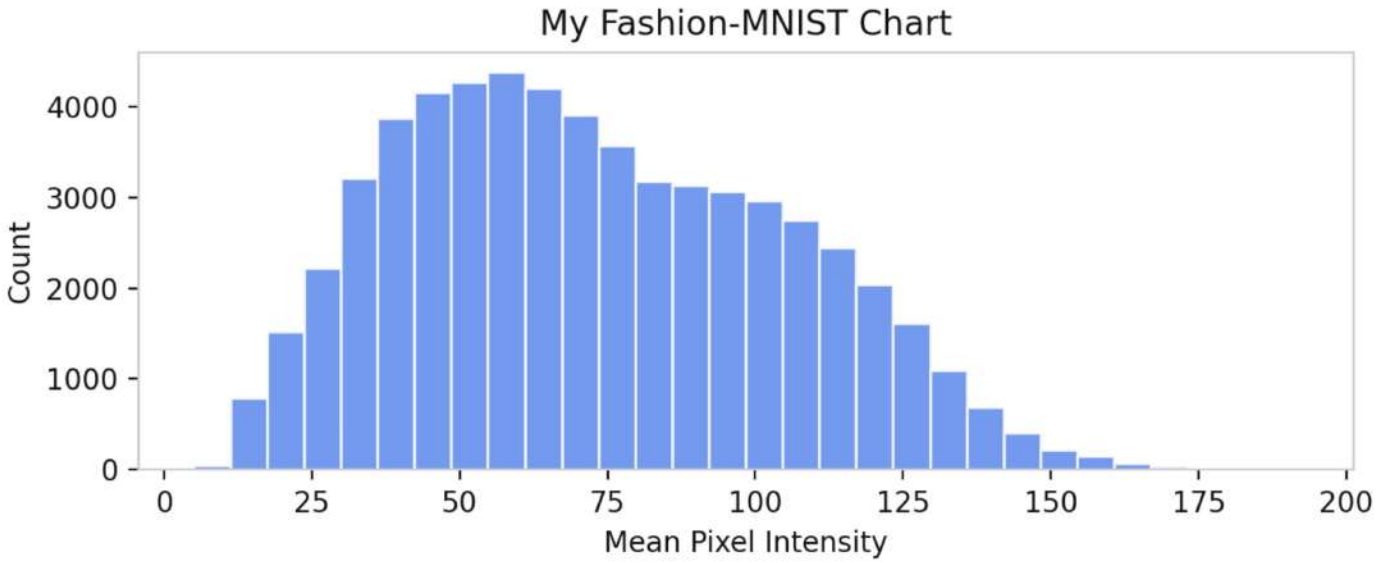
A major focus of the course is balancing the complexity of the data with the simplicity of design. This tutorial exemplifies that by working with a truly complex dataset (many images, many classes, many pixels) yet keeping the design simple (widgets only appear with a purpose).

```
title = st.text_input("Custom chart title", "My Fashion-MNIST Chart")
bins = st.number_input("Histogram bins", min_value=5, max_value=100, value=30, step=5)
```

Accent color

Select the color of the bars!

```
accent_color = st.color_picker("Accent color", "#4c8bf5")
```



Widget Showcase

Streamlit's widget system turns Python variables into interactive controls. Every widget returns a value and triggers a **full script rerun** when changed. Here we demonstrate some of the most popular widget types, applied to image exploration.

---

st.slider

Number of sample images to show

6

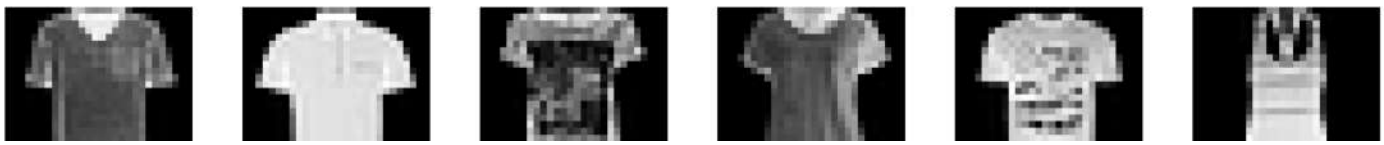
Brightness adjustment

0

```
n_images = st.slider("Number of sample images to show", 1, 20, 6)
brightness = st.slider("Brightness adjustment", -100, 100, 0,
                       help="Adds a constant to all pixel values")
```

Class to preview

T-shirt/top



`st.expander` — Collapsible Sections

`st.expander` hides content until the user clicks. Great for code snippets, details, or secondary information that would clutter the main view. You will see lots of expanders to show the code for each tool!

- 👕 T-shirt/top
- 👖 Trouser
- 🧥 Pullover
- 👗 Dress
- Expander Code

`st.tabs`

Tabs can be seen at the top of this application as well as a nested version below! Here we show per-class stats in nested tabs.

T-shirt/top | Trouser | Pullover | Dress | Coat

Count (train)	Mean pixel	Std pixel
6,000	83.0	89.4

- Tabs code

`st.dataframe` VS `st.table`

`st.dataframe` renders an **interactive sortable table** with column headers you can click. `st.table` renders a **static** table, which is better for small reference tables.

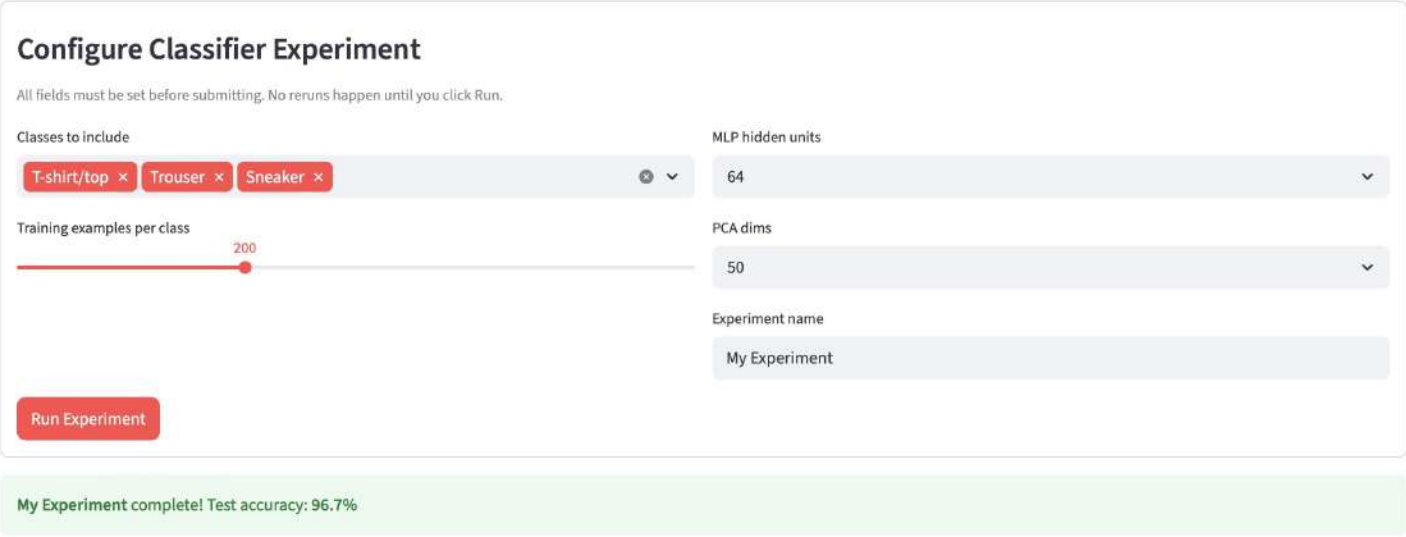
Interactive dataframe (vs static table)

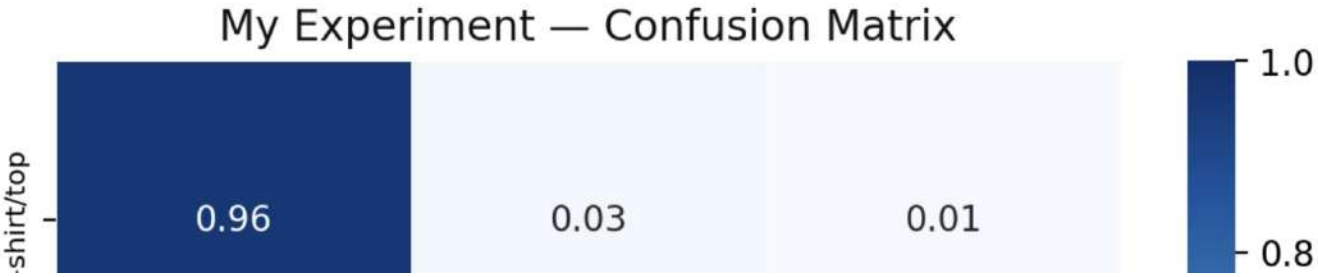
Class	Train count	Mean pixel	Std pixel
T-shirt/top	6000	83.03	89.44
Trouser	6000	56.84	87.6
Pullover	6000	96.06	91.46
Dress	6000	66.02	90.33
Coat	6000	98.26	95.96
Sandal	6000	34.87	67.09
Shirt	6000	84.61	86.52
Sneaker	6000	42.76	75.17
Bag	6000	90.16	93.14
Ankle Boot	6000	76.81	94.49

`st.dataframe` — click column headers to sort

> Dataframe & table code

Again, interactivity features can help to make the story more audience- and reader-driven. Cairo recommends adding interactivity so people can organize the data at will, similar to what you can do with the interactive dataframe above.


Configure Classifier Experiment
All fields must be set before submitting. No reruns happen until you click Run.
Classes to include
T-shirt/top ×
Trouser ×
Sneaker ×
Training examples per class
200
MLP hidden units
64
PCA dims
50
Experiment name
My Experiment
Run Experiment
My Experiment complete! Test accuracy: 96.7%


My Experiment — Confusion Matrix
-shirt/top
0.96
0.03
0.01
1.0
0.8

Individual Prediction Inspector

Select a filter below to browse specific types of predictions. The bar chart shows the full softmax distribution — not just the top prediction.

Show me:

Random images High-confidence errors Low-confidence correct

Images to display

6